



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Ćwiczenie: C12

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji.

Obsługa wyjątków w programowaniu obiektowym

Obsługa wyjątków to technika stosowana w programowaniu w celu zarządzania błędami, które mogą wystąpić podczas wykonywania programu. W szczególności w języku C# mechanizm ten pozwala na wykrywanie, przechwytywanie i obsługiwanie błędów, co pomaga w tworzeniu bardziej niezawodnych i stabilnych aplikacji.

Wyjątki w programowaniu obiektowym są specjalnymi obiektami, które reprezentują błędy lub niespodziewane warunki. W C# klasy wyjątków dziedziczą z klasy bazowej System.Exception.

Podstawowe pojęcia

Try: Blok kodu, w którym mogą wystąpić wyjątki.

Catch: Blok kodu, który przechwytuje i obsługuje wyjątki zgłoszone w bloku try.

Finally: Blok kodu, który zawsze jest wykonywany, niezależnie od tego, czy wystąpił wyjątek, czy nie.

Throw: Instrukcja używana do ręcznego zgłaszania wyjątków..

Przykład 1. Obsługa wyjątków przy parsowaniu ciągów na liczby

```
using System;
namespace Parsowanie
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Wprowadzenie danych wejściowych przez użytkownika
                Console.WriteLine("Podaj liczbę:");
                string wejscie = Console.ReadLine();

                // Próba parsowania ciągu na liczbę całkowitą
                int liczba = int.Parse(wejscie);
                Console.WriteLine($"Podano liczbę: {liczba}");
            }
        }
    }
}
```

```

    }
    catch (FormatException e)
    {
        // Obsługa wyjątku formatu
        Console.WriteLine("Wyjątek: Podany ciąg nie jest prawidłową liczbą.");
        Console.WriteLine($"Szczegóły wyjątku: {e.Message}");
    }
}
}
}

```

Przykład 2. Obsługa wyjątków przy dostępie do tablicy

```

using System;

namespace DostepDoTablicy
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                int[] liczby = { 1, 2, 3 };

                // Próba dostępu do elementu poza zakresem tablicy
                Console.WriteLine(liczby[10]);
            }
            catch (IndexOutOfRangeException e)
            {
                // Obsługa wyjątku dostępu poza zakresem tablicy
                Console.WriteLine("Wyjątek: Próba dostępu do elementu poza zakresem tablicy.");
                Console.WriteLine($"Szczegóły wyjątku: {e.Message}");
            }
        }
    }
}

```

Przykład 3. Obsługa wyjątków przy dzieleniu

```

using System;
namespace Dzielniki
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Wprowadzenie danych przez użytkownika
                Console.WriteLine("Podaj licznik:");
                int licznik = int.Parse(Console.ReadLine());
                Console.WriteLine("Podaj mianownik:");
                int mianownik = int.Parse(Console.ReadLine());

                // Próba dzielenia przez podany mianownik
                int wynik = licznik / mianownik;
                Console.WriteLine($"Wynik: {wynik}");
            }
            catch (DivideByZeroException e)
            {
                // Obsługa wyjątku dzielenia przez zero
                Console.WriteLine("Wyjątek: Nie można dzielić przez zero.");
                Console.WriteLine($"Szczegóły wyjątku: {e.Message}");
            }
        }
    }
}

```

Przykład 4. Obsługa wyjątków własnych

```

using System;
namespace WyjatkiWlasne
{
    // Definicja własnego wyjątku
    public class MojWyjatek : Exception
    {

```

```

        public MojWyjatek(string message) : base(message) { }
    }

    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Zgłoszenie własnego wyjątku
                throw new MojWyjatek("To jest mój własny wyjątek!");
            }
            catch (MojWyjatek e)
            {
                // Obsługa własnego wyjątku
                Console.WriteLine("Wyjątek: " + e.Message);
            }
        }
    }
}

```

Zadania do samodzielnego rozwiązania

Zadanie 1. Rozwiń przykład 1, tak aby prosił użytkownika o podanie dwóch liczb i próbował je dodać. Użyj obsługi wyjątków, aby obsłużyć błędy związane z niepoprawnym formatem danych wejściowych.

Zadanie 2. Rozwiń przykład 2, tak aby prosił użytkownika o podanie indeksu i próbował zmienić element tablicy znajdujący się pod tym indeksem. Użyj obsługi wyjątków, aby obsłużyć błędy dostępu poza zakresem tablicy.

Zadanie 3. Rozbuduj przykład 3, dodając dodatkowy blok catch, który przechwyci wyjątek `FormatException`, gdy użytkownik poda nieprawidłowy format liczby podczas wprowadzania danych.

Zadanie 4. Stwórz klasę wyjątku, która dziedziczy po `Exception`, i napisz program, który zgłasza wyjątek, gdy liczba podana przez użytkownika jest ujemna. Użyj tej klasy wyjątku do przechwycenia i obsługi błędu w programie.